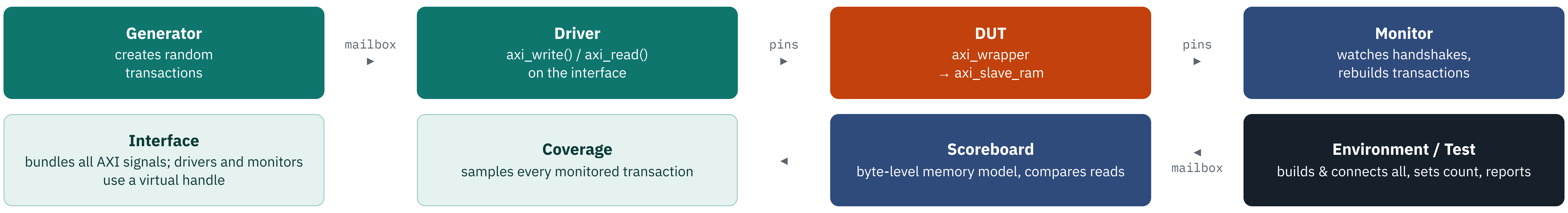
DV INTERVIEW · GLANCE SHEET 11 · PROJECT FINAL PART

Verifying the AXI RAM: Full Testbench

From hand-written tasks to a layered, random, self-checking environment with coverage — the same shape as UVM.

generate → drive → monitor → check
random + constrained + covered
done = 0 errors & 100% coverage

1 Layered testbench architecture



Generator
creates random transactions

mailbox ▸

Driver
axi_write() / axi_read() on the interface

pins ▸

DUT
axi_wrapper → axi_slave_ram

pins ▸

Monitor
watches handshakes, rebuilds transactions

Interface
bundles all AXI signals; drivers and monitors use a virtual handle

Coverage
samples every monitored transaction

◂

Scoreboard
byte-level memory model, compares reads

◂ mailbox

Environment / Test
builds & connects all, sets count, reports

Why layers? Each block has one job, so you can swap the generator (new test) without touching the driver, or reuse the monitor elsewhere.

The monitor is passive — it never drives a pin. The scoreboard trusts the monitor, not the driver, so driver bugs don't hide DUT bugs.

The driver reuses the `axi_write` / `axi_read` tasks from the previous two sheets.

2 Transaction class & constraints

```
class axi_txn;
  rand bit        wr;            // 1 = write, 0 = read
  rand bit [3:0]  id;
  rand bit [31:0] addr;
  rand bit [7:0]  len;
  rand bit [2:0]  size;
  rand bit [1:0]  burst;
  rand bit [31:0] data[];
  rand bit [3:0]  strb[];
       bit [1:0]  resp;          // filled by monitor

  constraint c_legal {
    burst != 2'b11;
    size  <= 3'b010;                     // 32-bit bus
    burst == 2'b10 -> len inside {1, 3, 7, 15};
    burst == 2'b00 -> len < 16;
    burst == 2'b10 -> addr % (1 << size) == 0;
  }
  constraint c_range {
    addr < DEPTH * 4;
    burst == 2'b01 ->
      (addr % 4096) + (len + 1) * (1 << size) <= 4096;
    burst == 2'b01 -> addr + (len + 1) * (1 << size) <= DEPTH * 4;
  }
  constraint c_arrays {
    data.size() == len + 1;
    strb.size() == len + 1;
  }
  constraint c_dist {
    len dist {0 := 20, [1:15] :/ 50, [16:255] :/ 30};
    solve burst before len;
  }
endclass
```

3 Environment & test

```
class axi_env;
  generator  gen;   driver     drv;
  monitor    mon;   scoreboard sb;
  mailbox #(axi_txn) gen2drv = new();
  mailbox #(axi_txn) mon2sb  = new();

  function new(virtual axi_if vif);
    gen = new(gen2drv);
    drv = new(gen2drv, vif);
    mon = new(mon2sb,  vif);
    sb  = new(mon2sb);
  endfunction

  task run(int n);
    gen.count = n;
    fork
      gen.run();  drv.run();
      mon.run();  sb.run();
    join_none
    wait (sb.checked == n);       // every txn seen & checked
    sb.report();                  // PASS only if errors == 0
  endtask
endclass
```

DRIVER LOOP

```
task run();
  forever begin
    gen2drv.get(t);
    if (t.wr) axi_write(t);   // from sheet 9
    else      axi_read (t);   // from sheet 10
  end
endtask
```

End of test = scoreboard has checked N transactions, not "generator finished" — otherwise the last responses are never compared.

4 Monitor

```
task run();
  forever begin
    @(posedge vif.ACLK);
    if (vif.AWVALID && vif.AWREADY)
      wr = capture_aw();            // addr, len…
    if (vif.WVALID && vif.WREADY) begin
      wr.data.push_back(vif.WDATA);
      wr.strb.push_back(vif.WSTRB);
    end
    if (vif.BVALID && vif.BREADY) begin
      wr.resp = vif.BRESP;
      mon2sb.put(wr);               // write complete
    end
    // same pattern for AR / R → put on RLAST
  end
endtask
```

- Samples **only on handshake edges**.
- Sends a write only after B, a read only after RLAST.
- Also a good home for protocol checks (or bind SVA).

5 Scoreboard

```
bit [7:0] mem_model [bit [31:0]];   // sparse bytes

task run();
  forever begin
    mon2sb.get(t);
    a = t.addr;
    for (int b = 0; b <= t.len; b++) begin
      for (int l = 0; l < 4; l++) begin
        ba = (a & ~32'h3) + l;          // byte address
        if (!in_beat(a, t.size, l)) continue;
        if (t.wr && t.strb[b][l])
          mem_model[ba] = t.data[b][8*l +: 8];
        else if (!t.wr && mem_model.exists(ba) &&
                 t.data[b][8*l +: 8] !== mem_model[ba])
          error(t, b, l);
      end
      a = next_addr(a, t.len, t.size, t.burst);
    end
    checked++;
  end
endtask
```

Byte-level model handles narrow, unaligned and partial-strobe writes for free. `next_addr()` is the same function as the RTL — but write it **independently** in the TB, or a shared bug passes.

6 Functional coverage

```
covergroup axi_cg with function sample(axi_txn t);
  cp_dir   : coverpoint t.wr;
  cp_burst : coverpoint t.burst
             { bins b[] = {0, 1, 2}; }
  cp_len   : coverpoint t.len {
    bins one     = {0};
    bins short_  = {[1:15]};
    bins sixteen = {15};
    bins long_   = {[16:254]};
    bins max     = {255}; }
  cp_size  : coverpoint t.size { bins s[] = {[0:2]}; }
  cp_align : coverpoint (t.addr % (1 << t.size) == 0);
  cp_strb  : coverpoint t.strb[0] {
    bins full = {4'hF}; bins none = {4'h0};
    bins part = default; }
  x_bls    : cross cp_burst, cp_len, cp_size;
  x_dir    : cross cp_dir, cp_burst;
endgroup
```

Plus **protocol coverage** sampled in the monitor: READY back-pressure on each channel, W-before-AW, read and write overlapping.

7 Test plan

Test	How	Targets
sanity	Directed single write + read	Bring-up, connectivity
burst_types	Each of FIXED / INCR / WRAP, write then read	Address generator
cross_burst	Write WRAP, read INCR and vice versa	Address order bugs
strobe	Random WSTRB, narrow & unaligned	Byte enables, lanes
backpressure	Random BREADY / RREADY delays	Hold & stable rules
concurrent	Write and read threads in parallel	FSM independence
max_len	LEN 255 INCR at a 4 KB-aligned base	Counters, RLAST
random	N = 10 000 random txns, many seeds	Everything; closes coverage
reset	Assert ARESETn mid-burst, then resume	Reset recovery
error	Out-of-range address, BURST = 11	SLVERR handling

8 Bugs this TB is built to catch

Bug	Caught by
WRAP wraps at wrong boundary	cross_burst + scoreboard
Beat-2 address not aligned	strobe (unaligned) + scoreboard
WSTRB ignored	strobe + byte model
RLAST off by one	axi_read RLAST check
RDATA changes while stalled	backpressure + SVA
BID / RID wrong	Driver ID checks
Read blocks while writing	concurrent (timeout)
Counter overflow at LEN 255	max_len
Stuck after reset	reset test

✓ Sign-off checklist

☐ All tests pass on many seeds, **0 errors**
☐ **Functional coverage 100%** of the plan (or waivers justified)
☐ **Code coverage** reviewed: line, branch, toggle, FSM states & transitions
☐ All **SVA** protocol assertions enabled and passing
☐ No X on outputs after reset
☐ Every bug filed, fixed and re-run in regression
☐ Scoreboard actually compared N transactions (no silent pass)

100% coverage with 0 checks is a classic trap — always confirm the checker ran.

U Same TB in UVM

This TB	UVM
axi_txn	uvm_sequence_item
generator	uvm_sequence + uvm_sequencer
driver	uvm_driver
monitor	uvm_monitor + analysis port
mailbox	TLM port / analysis FIFO
scoreboard	uvm_scoreboard
driver + monitor + sqr	uvm_agent
env	uvm_env
test / N	uvm_test + config_db
wait(checked == n)	objections

? Interview Q&A

Question	Answer
Why a monitor if the driver knows what it sent?	Driver could be wrong; monitor shows what really happened on the pins.
Why a byte-level model?	Handles strobes, narrow and unaligned without special cases.
How do you know you're done?	Coverage closed, 0 errors over seeds, code coverage reviewed.
Random or directed?	Directed for bring-up, constrained random to close coverage.
Biggest risk in this TB?	Sharing address logic with RTL — common-mode bugs pass.
How to end the test cleanly?	Wait for scoreboard count (UVM: objections).